Phase 4–5 记录：可审计筛查器与两级半径消融

日期：2026-10-11 · **对应**：handout Phase 4（最小方法）、Phase 5（完整设计的一部分）、RQ3

代码：`scripts/screening.py`（模块 + 9 项单元测试）、`scripts/featurize.py`（特征缓存）

1. 模块做到了什么

handout 要求	实现	证据
观测中心取自 M	贪心核心集，中心是 M 中的实际向量	`ScreenIndex.__init__`
保守半径 r_j ≥ max d(m,c_j)	`assign_and_radii` 按 cell 取最大距离	单元测试 1/2
`L ≤ s ≤ U`	`bounds()` + `image_bounds()`	单元测试 1/2/5
不确定就升级	`U≤t→正常，L>t→报警，否则升级`	单元测试 3
整库不可用/版本不符 → 未决	`Resolver.resolve` 返回 `unresolved:*`	单元测试 6
版本契约	`BankVersion.id()` = 编码器+预处理+库哈希+阈值+聚合+K+层数 的哈希	单元测试 6
元数据计量	`bytes()` 返回中心/半径/元数据三段字节	消融表最后一列
确定性	两次构建逐位一致	单元测试 7
两级细化	大 cell 切子 cell；子中心 = 父中心 ∪ medoid	单元测试 5

手写单元测试 9 项全部通过（并列、重复向量、空 cell、阈值相等、两级、全局半径、版本不符、离线、确定性）。

2. 单元测试抓到的两个真 bug（都已修）

1. **子中心用了 k-means 均值 → `L ≤ s ≤ U` 失效。**

`U(q)=min_j d(q,c_j)` 要成为上界，前提是 `c_j ∈ M`；均值不在 M 里，会让 U 小于真值。

→ 改为 **medoid**（簇内离质心最近的实际向量）。

2. **子中心不保留父中心 → U 反而变差**（区间 5.696 → 5.890）。

→ 子中心集合改为 **{父中心} ∪ {medoids}**，实测区间 5.696 → 5.270（更紧）。

这两条是"看起来只是实现细节、实际破坏数学前提"的典型，值得写进论文的数值有效性一节。

3. 真实数据消融（16-shot，库 12,544 patch，阈值=校准集 95 分位）

3.1 k 由压缩比决定（k=1254 ≈ 10%）

类别	层	半径	本地率	升级率	错判	字节 KB
bottle	1	cell	89.2%	10.8%	0	7529
bottle	2	cell	89.2%	10.8%	0	7589
tile	1	cell	38.5%	61.5%	0	7529
tile	2	cell	38.5%	61.5%	0	7589
screw	1	cell	84.4%	15.6%	0	7529
screw	2	cell	84.4%	15.6%	0	7811

观察：二级几乎无收益。原因不是实现错误，而是**平均 cell 只有 10 个点**（12,544/1,254），远低于切分阈值 64 —— 两级结构在细粒度下无事可做。字节数略增说明仅极少数大 cell 被切。

3.2 强制粗粒度（k=32）

类别	层	半径	本地率	升级率	错判	字节 KB
bottle	1	cell	1.2%	98.8%	0	192

类别	层	半径	本地率	升级率	错判	字节 KB
bottle	2	cell	1.2%	98.8%	0	576
tile	1	cell	1.7%	98.3%	0	192
tile	**2**	**cell**	**26.5%**	73.5%	0	504
screw	1	cell	0.0%	100.0%	0	192
screw	**2**	**cell**	**9.4%**	90.6%	0	552
bottle	1	global	0.0%	100.0%	0	192
tile	2	global	25.6%	74.4%	0	504

三个结论

1. **两级细化在"区间是约束"的类别上有效**：tile 1.7%→26.5%（+24.8 pp），screw 0.0%→9.4%。

2. **在"检测器余量大"的类别上无效**：bottle 一直 1.2%，因为它 AUROC=1.0，决策远离阈值；

细化省下的宽度用不上。

3. **全局半径 vs 逐 cell 半径**：差距小（bottle 1.2% vs 0%；tile 二级 26.5% vs 25.6%），

说明**半径的紧度主要来自 cell 划分本身，而不是"是否逐 cell 取半径"**。这是个反直觉但可解释的结果。

代价：两级把中心数从 32 扩到约 100，字节 192 → 504–576 KB（约 3×）。

4. 与 Phase 3 数字的一致性

Phase 3（`w3_decision_16shot.json`）在 10% 压缩下 bottle 的升级率是 8.4%，本模块给出 10.8%。

差异来源：Phase 3 的 `screen_scores` 用 GPU `cdist` 且 radii 的浮点保护量不同；本模块用 `GUARD=1e-6`。

两者都在 8–11% 量级，方向一致；但**同一实验的两个实现出现 2.4 pp 差异，必须在报告里说明**，

Phase 6 应统一到同一个实现（以本模块为准）并重跑 Phase 3 的筛查列。

5. 待办（Phase 5–9）

项	状态
断网/版本不符的端到端演示（真实数据）	单测覆盖，未做端到端
批处理吞吐与时延（p50/p95）	未做
精确树基线（BallTree/cover-tree）对照	未做
多种子、多参考数量的主网格	未做
位移/微小缺陷/失败图库	未做
分析笔记（证明、复杂度、盈亏平衡）	未做
论文草稿	未做

6. 复现

```
python scripts\featurize.py --categories bottle tile screw         # 特征缓存（约 7 分钟）
python scripts\screening.py --selftest                             # 9 项单测
python scripts\screening.py --category all --ratio 0.10 --levels 1 2 --radius cell global
python scripts\screening.py --category all --k 32 --levels 1 2 --radius cell global --out
logs\w4_screening_k32.json
```